

Automatic Detection of Scoria Cones from DEMs

Gabriele Berruti (4791919)
Computational Vision — University of Genoa

1 Introduction

This project detects *scoria cones* from single-channel Digital Elevation Models (DEMs), where each pixel encodes a terrain elevation in metres.

The core contribution is the **separation of nested or adjacent cones**: point-based blob detectors collapse two overlapping craters into a single response, and a one-to-one matching then discards one of the two.

We address this with a watershed segmentation of the crater depressions, which natively splits adjacent catchment basins.

We work on three datasets: **El Hierro** (5 m resolution, with ground-truth annotations, used for development and validation), **Etna** (2 m, noisier ground truth, used as a stress test) and **Jeju** (10 m, no ground truth, used for qualitative and unsupervised analysis).

2 Data and preprocessing

2.1 Step 0 — DEM loading and inspection

A DEM is a *range image*: a single elevation band.

Each dataset uses a different *nodata* sentinel, which we convert to **NaN** so that all later computations ignore invalid pixels naturally.

Table 1 summarises the three DEMs after loading.

In Table 1, $H \times W$ is the raster size in pixels (matrix rows $\times$ columns), Mpx the total number of pixels in millions ($H \cdot W$, a proxy for the memory footprint), NaN% the share of pixels that were *nodata* and got mapped to **NaN** (terrain outside the raster/island, ignored by every later step), Res. the ground size of one pixel in metres, dtype the numeric type of the elevation values, and $z_{\max}$ the maximum elevation (sanity check against the real volcano).

Dataset	Size (H $\times$ W)	Mpx	NaN%	Res. (m)	dtype	$z_{\max}$ (m)
El Hierro	5081 $\times$ 5681	28.9	40%	5	float32	1499.8
Etna	27756 $\times$ 25717	713.8	25%	2	float32	3324.1
Jeju	4364 $\times$ 7761	33.9	27%	10	float32	1945.0

Table 1: DEM inspection. Maximum elevations match the real volcanoes (Etna $\approx$ 3.3 km, El Hierro $\approx$ 1.5 km, Hallasan/Jeju $\approx$ 1.95 km), confirming correct loading.

Three facts drive the following steps: (i) Etna is ≈ 714 Mpx (≈ 2.86 GB as float32), so it must be processed in *tiles* rather than loaded whole;

(ii) the three *nodata* conventions differ (-3.4×10^{38} , -9999 , NaN) and are all mapped to NaN ;

(iii) each dataset has its own UTM coordinate reference system, so any shapefile must be reprojected into the CRS of its DEM before use.

2.2 Step 0.5 — Visual inspection

Before deriving any channel we inspect the data on El Hierro.

Figure 1 shows that the raw DEM looks visually flat, whereas the Eduard renderings (NE/NW shading and texture shading) reveal the cones; this motivates the derived channels of Step 1.

Figure 2 shows the 196 crater outlines of the ground truth: the two closest craters lie only 25 m apart (5 pixels) and several cones are adjacent or nested.

Point-based blob detectors merge such pairs into a single detection — separating them is the goal of the watershed step.

For clarity, the panels in Figure 1 are: the **raw DEM**, i.e. the original elevation grid where each pixel is a terrain height in metres (it looks flat because a cone is only tens of metres tall relative to the whole island);

the **Eduard renderings**, relief-shaded views produced by the *Eduard* relief-shading tool and shipped with the dataset;

the **NE/NW shading** (hillshade), where the terrain is lit by a simulated sun from the North-East/North-West so that slopes facing the light are bright and those facing away are dark (two opposite directions are given so no slope stays in shadow);

and the **texture shading**, a light-direction-independent rendering that emphasises fine surface detail.

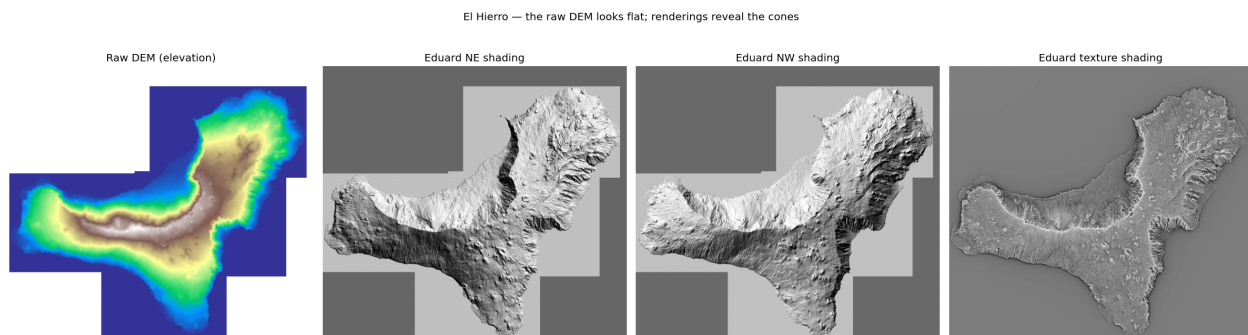


Figure 1: El Hierro: the raw DEM (left) is visually flat, while the Eduard renderings reveal the conical landforms.

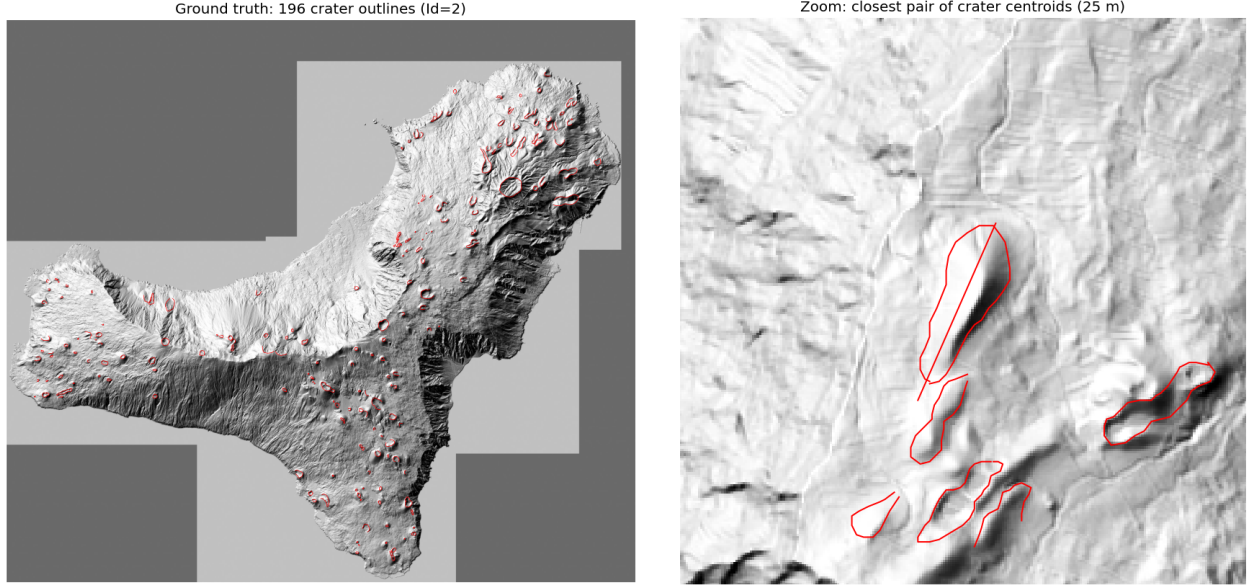


Figure 2: Ground truth on El Hierro. Left: the 196 crater outlines (Id= 2). Right: a zoom on the closest pair of crater centroids (25 m apart).

Figure 3 isolates the cones that actually *overlap*, i.e. whose nearest neighbour is closer than the sum of the two radii.

These are 8 craters in 4 pairs; note that they are not the single closest centroids of Figure 2 (two small craters), but pairs of *large* adjacent cones whose outlines intersect.

These are the configurations a point-based detector merges into one and the watershed step must separate.

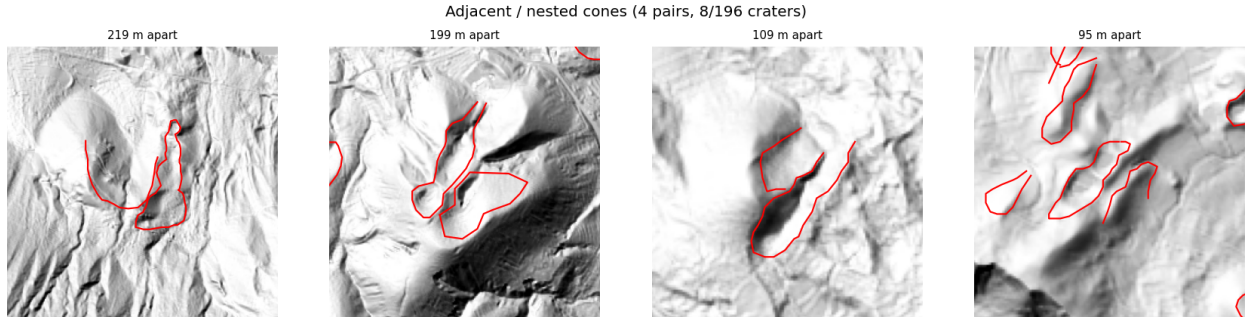


Figure 3: The adjacent/nested cases on El Hierro: the 4 crater pairs whose outlines overlap (8 craters). The watershed step is designed to split exactly these.

Ground-truth quality and label semantics. The El Hierro shapefile is *dirty* and mixes three feature types in a single file.

The crater outlines (Id= 2) are open polylines: 191 of 196 do not close (median start–end gap 40 m, up to 556 m) and some have as few as 2 vertices.

We therefore use crater *centroids* (plus an approximate radius) rather than the exact outline shape.

A geometry test then characterises the three labels (Table 2): Id= 1 is the cone **base** (large, round, and enclosing the crater in 98% of cones), Id= 2 the **crater** rim, and Id= 3 a **fissure** — a short, strongly elongated segment that always carries a non-zero azimuth in the `Prj_Az` field.

The `pericolosi` field is a hazard level (0/1/2).

We adopt Id= 2 as the detection target.

Id	meaning	n	verts	length (m)	aspect	Prj_Az≠ 0
1	base	151	29	846	1.29	0%
2	crater	196	19	457	1.47	0%
3	fissure	171	2	272	8.32	100%

Table 2: Data-driven characterisation of the El Hierro labels (median values). The high aspect ratio and the always-present azimuth identify Id= 3 as fissures.

Observation — what these data fix for the pipeline. The geometry tests turn the three labels into three decisions.

(i) The **crater** (Id= 2) is the detection target: it is the only label present on *every* cone (196), and since its outlines are open we represent each crater by its **centroid and equivalent radius** rather than by the polygon.

(ii) The **base** (Id= 1) encloses the crater in 98% of cones — this containment is exactly the nested configuration our contribution must preserve, and it gives a free sanity check on a detection.

(iii) The **fissure** (Id= 3), a short segment with a real azimuth, is not a target but feeds the qualitative cone–fissure alignment of the bonus step.

Finally, the labels are *not* equinumerous (151 base / 196 crater / 171 fissure): annotations are incomplete, so the whole pipeline is anchored on the complete crater set.

2.3 Step 0.6 — Quick data analysis

Three cheap measurements on El Hierro parameterise the later steps rather than guessing their constants.

(i) The three Eduard renderings share the DEM grid exactly (same 5081×5681 shape and transform), so overlays are pixel-exact.

(ii) The equivalent radius of the crater outlines (Figure 4, left) has median 71 m and a 5th–95th percentile range of 20–174 m, which we adopt as the LoG detector scale range.

(iii) The nearest-neighbour distance between craters (Figure 4, right) has median 418 m but a minimum of 25 m, and 8/196 craters (4%) lie closer to a neighbour than the sum of the two radii, i.e. they are adjacent or nested.

This 4% is precisely the population that point-based detection merges and that the watershed step is designed to recover.

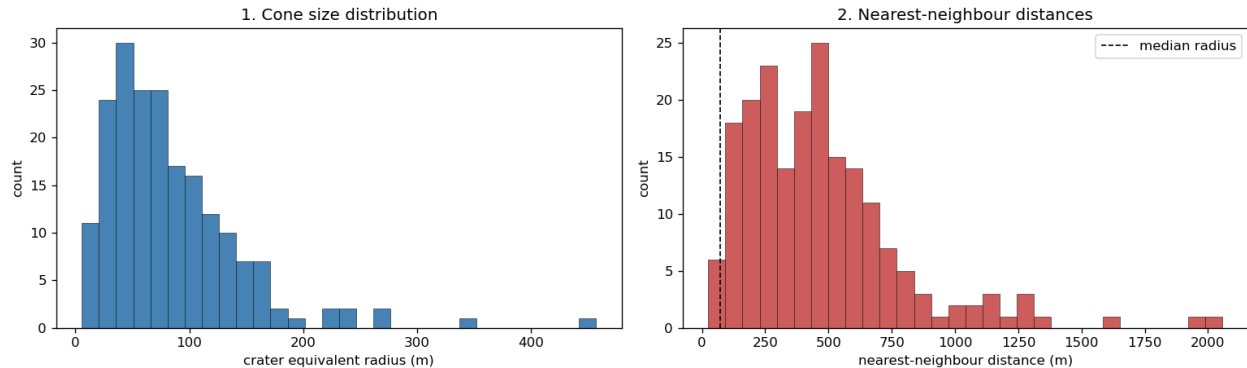


Figure 4: El Hierro crater statistics. Left: equivalent-radius distribution, used to set the LoG scale range. Right: nearest-neighbour distance distribution; the dashed line is the median radius, and the cones below it are the adjacent/nested cases.

Observation — what these three checks fix. Put together, the measurements set three things at once.

The **alignment** lets every DEM-derived channel and the supplied shadings be stacked as co-registered inputs with no resampling; the crater **sizes** fix the LoG search range to 20–174m (so the detector looks only at these radii); and the **adjacency** count shows that only 4% of cones overlap, yet this 4% is exactly the population a point-based detector merges into one and that the watershed step must recover.

In short these numbers parameterise the detector (scale range), guarantee clean multi-channel inputs (alignment), and size the problem the contribution must solve (the 4% adjacent/nested cones).

3 Detection pipeline

3.1 Step 1 — Morphometric channels

From the single elevation band we derive three channels with the image gradient (deck 01_image_processing, p.13): the **slope** = $\arctan \sqrt{g_x^2 + g_y^2}$ in degrees, the **aspect** = $\text{atan2}(g_y, g_x)$ (the direction the slope faces), and the **roughness**, the local standard deviation of elevation in a 5×5 window.

Figure 5 shows them on El Hierro.

As a quantitative check, the median slope on the crater rims is 18.8° against 8.1° on the background (about $2.3\times$ steeper), and the aspect rotates around each cone (bottom-right panel).

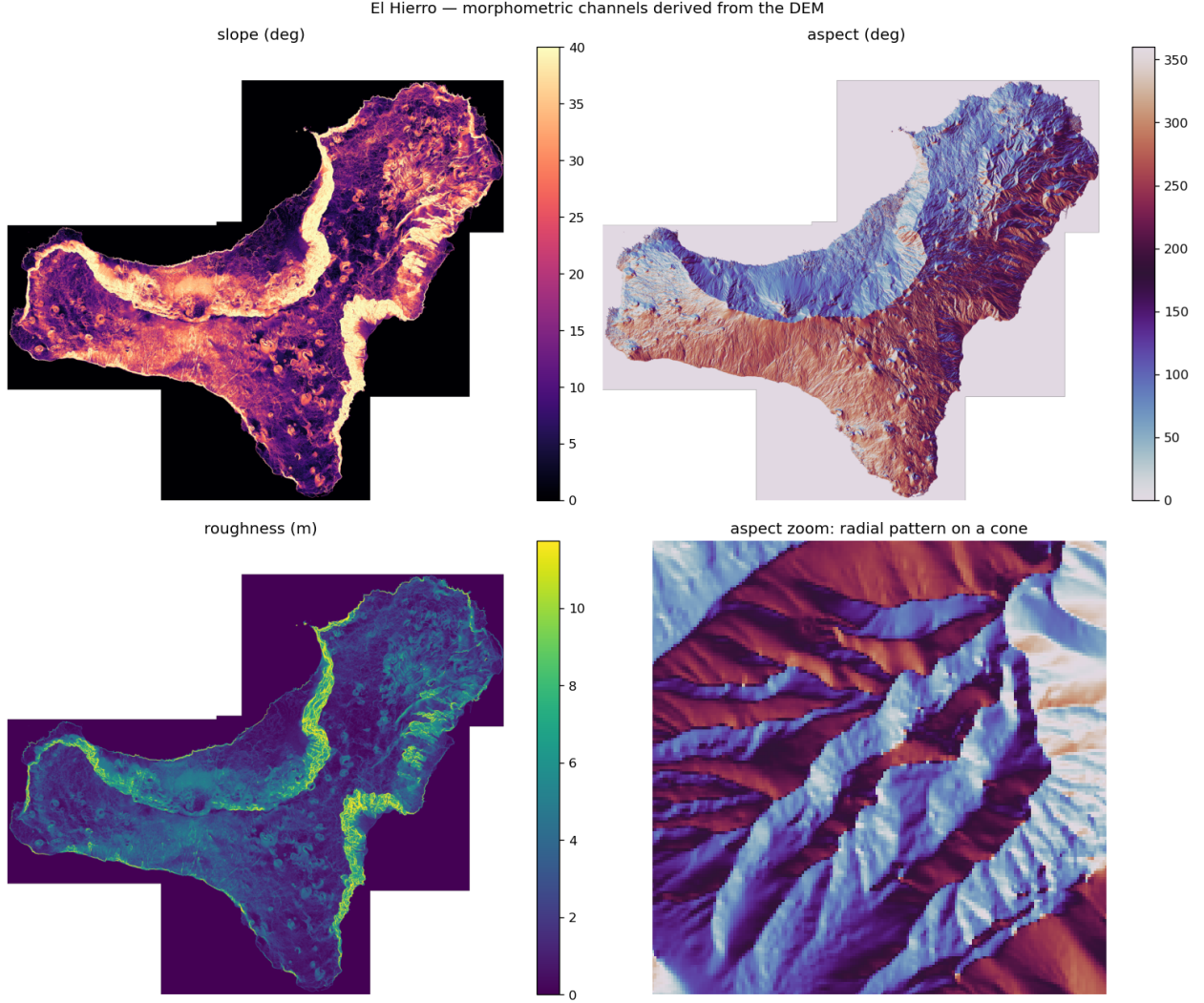


Figure 5: Morphometric channels derived from the El Hierro DEM: slope, aspect and roughness, plus a zoom of the aspect showing the radial pattern around a single cone.

Cones thus appear as bright *rings* in slope/roughness and as radial *colour wheels* in aspect; the only confounder is the coastal cliff, as steep as a crater rim, which is why Step 3 combines slope with roughness rather than thresholding slope alone.

These three channels, together with the texture rendering, are the input of the detection steps.

3.2 Step 2 — Synthetic hillshade from steerable filters

Rather than rely on the supplied Eduard NE/NW renders, we *synthesise* the hillshade from any light azimuth ourselves, using **steerable first-derivative-of-Gaussian filters** (deck 01_image_processing, p.22).

The DEM is filtered *once* along x and y to obtain the two basis responses $G_1^{0^\circ}$ and $G_1^{90^\circ}$; the derivative along an arbitrary direction θ is then a cheap linear combination, with no re-filtering:

$$G_1^\theta = \cos \theta G_1^{0^\circ} + \sin \theta G_1^{90^\circ}. \quad (1)$$

The shading lit from azimuth θ is the directional derivative along the light (bright where the surface rises toward it)¹.

With $\sigma = 2$ px (≈ 10 m) the synthetic NE hillshade reproduces the supplied Eduard NE render with a correlation of 0.81 (Figure 6), and rotating θ swings the illumination coherently (NE / NW / SE panels).

This yields a CV-grounded, self-produced multi-direction shading we fully control — unlike the two fixed renders — from only two precomputed filters.

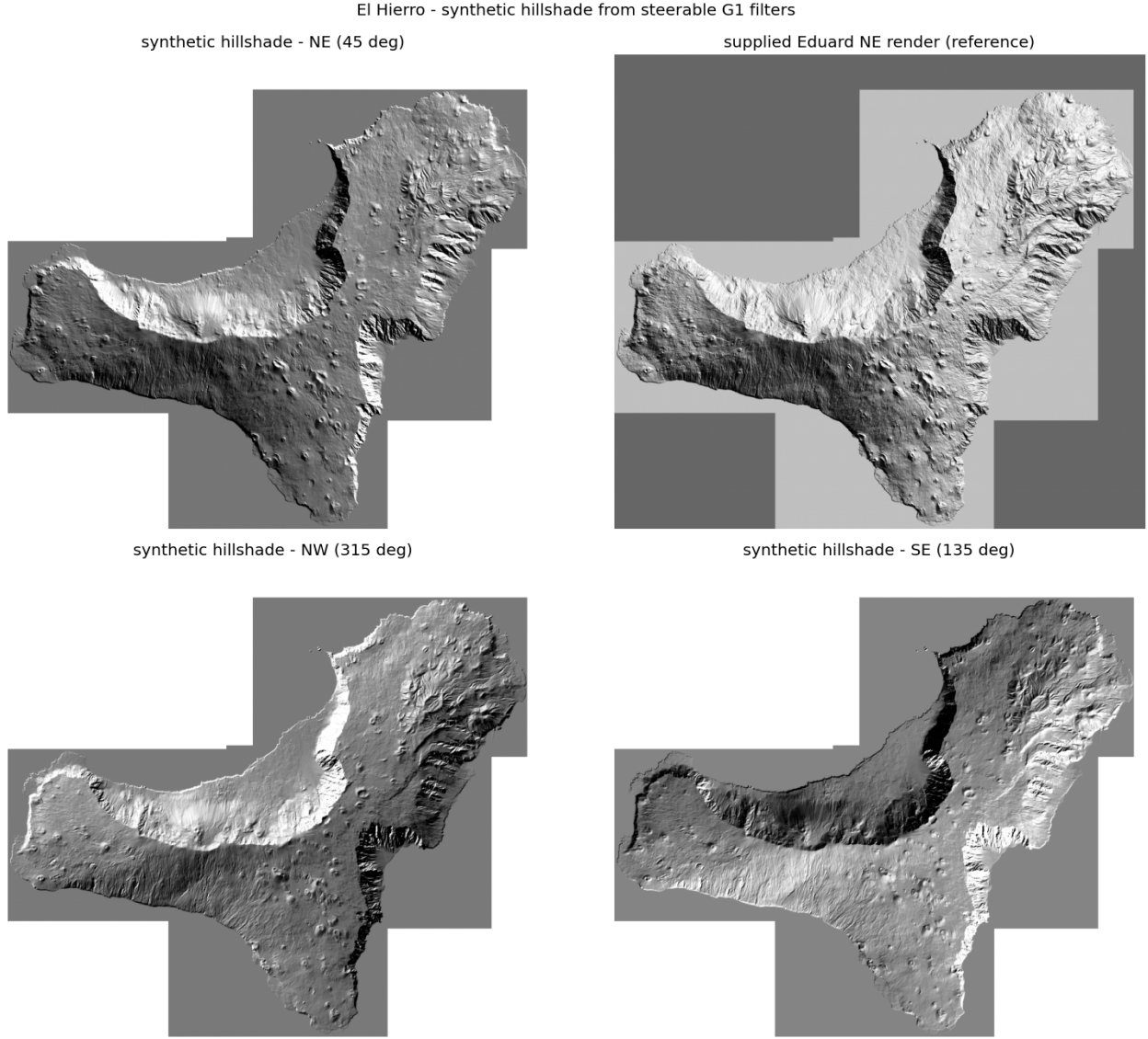


Figure 6: Synthetic hillshade on El Hierro from the steerable G_1 filters. Top: synthetic NE (45°) next to the supplied Eduard NE render (correlation 0.81). Bottom: the same basis re-steered to NW (315°) and SE (135°), showing the illumination rotating coherently.

¹Tested: with the raw directional derivative the synthetic shading comes out inverted relative to the Eduard render (correlation -0.81), as our light-direction convention is opposite to theirs; negating it ($s = -d$) flips the sign to $+0.81$.

3.3 Step 3 — Data-driven morphometric mask

To shrink the search area for the blob detector (Step 4) we keep only the **steep terrain** where cones live, with a binary mask obtained by **thresholding the slope** (deck 09_segmentation, p.13–18).

The threshold is not hand-picked: **Otsu’s method** selects the global optimum that best separates the slope histogram into *flat* and *steep* (maximising the between-class variance).

Notably it lands at 17.6° , right on the rim slope measured in Step 1 (18.8°), confirming the cut is meaningful.

The original plan used a fixed band $[25^\circ, 35^\circ]$, but on the real data that misses most rims (rim-recall 21%), which is exactly why we switched to a data-driven threshold; a roughness AND-term was also tested but gave no gain, since cone rims are already both steep and rough.

The resulting mask keeps only 31% of the island yet covers 96% of the cones (a cone counts as covered when steep terrain fills at least 10% of its disk), so it is a cheap, high-recall pre-filter (Figure 7).

Its one blind spot is the coastal cliff, which is steep and therefore survives the mask; it will be rejected later by the classifier (Step 7).

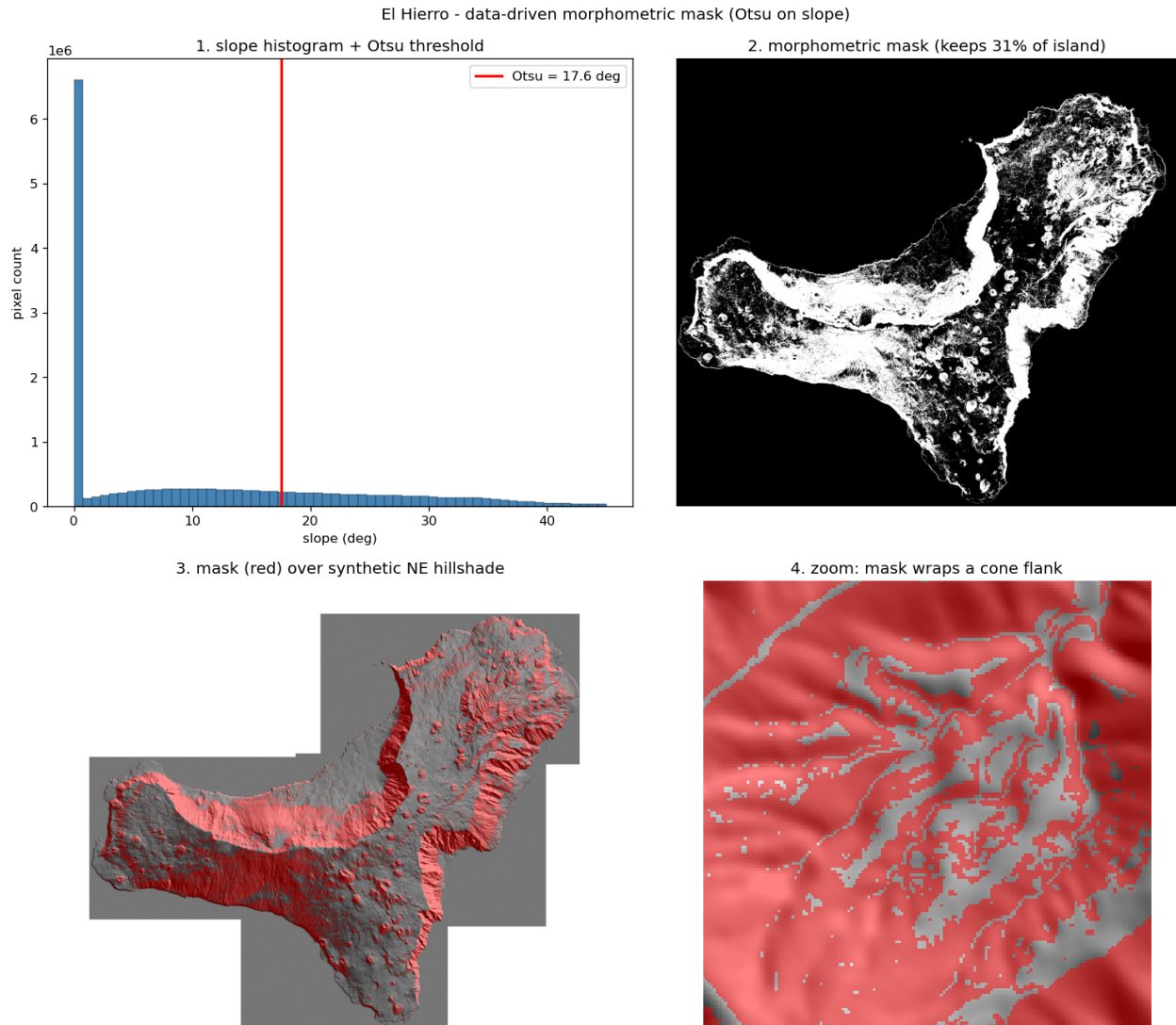


Figure 7: Data-driven morphometric mask on El Hierro. (1) Otsu threshold on the slope histogram (17.6°); (2) the resulting binary mask, keeping 31% of the island; (3) the mask (red) over the synthetic NE hillshade; (4) a zoom showing the mask wrapping a single cone flank.

3.4 Step 4 — LoG candidate detection (baseline that exposes the problem)

Terminology. A few terms recur from here on.

The *ground truth* (GT) is the set of annotated true craters.

A *true positive* (TP) is a real cone correctly detected, a *false positive* (FP) a detection matching no real cone, a *false negative* (FN) a real cone missed.

$Recall = TP / (TP + FN)$ measures how complete the detection is (what fraction of real cones we find), $precision = TP / (TP + FP)$ how clean it is (what fraction of detections are real).

A *blob* is the roughly circular spot the LoG returns at a given scale; a *one-to-one assignment* forces one detection per cone.

The *Random Forest* (RF) is the supervised ensemble-of-trees classifier of Step 7, and the *watershed* is the region-based segmentation of Step 5 that floods the surface from markers and separates adjacent basins.

On the masked texture rendering we run the **Laplacian-of-Gaussian** blob detector (`skimage.feature.blob_log`), the classic **scale-invariant** circular-feature detector (deck 03_image_matching, p.14–16).

The scales are taken from the radii measured in Step 0.6 (20–174 m) and the configuration is kept *deliberately permissive* (low threshold) to maximise recall: this is a high-recall candidate pool, not the final detector.

Against the 196 ground-truth craters (200 m matching tolerance) it yields 22,065 candidates with **recall** 0.964 (189/196, the 7 misses being small or eroded cones) and **precision** 0.009.

The recall reproduces the reference notebook exactly; the thousands of false positives are expected at this stage and will be removed by the Random Forest of Step 7.

The decisive observation concerns the adjacent cones.

Under the lenient *any-candidate* rule both members of a crowded pair count as found, but a real detector must output **one detection per cone**.

Enforcing a one-to-one assignment, only 186/196 cones keep a unique detection: 10 **cones** (the reference reports 11) are lost because neighbours compete for the same candidate blobs (Figure 8, right).

Point-based blob detection cannot separate them — this is precisely the $\approx 5\%$ gap that the watershed of Step 5 is built to close.

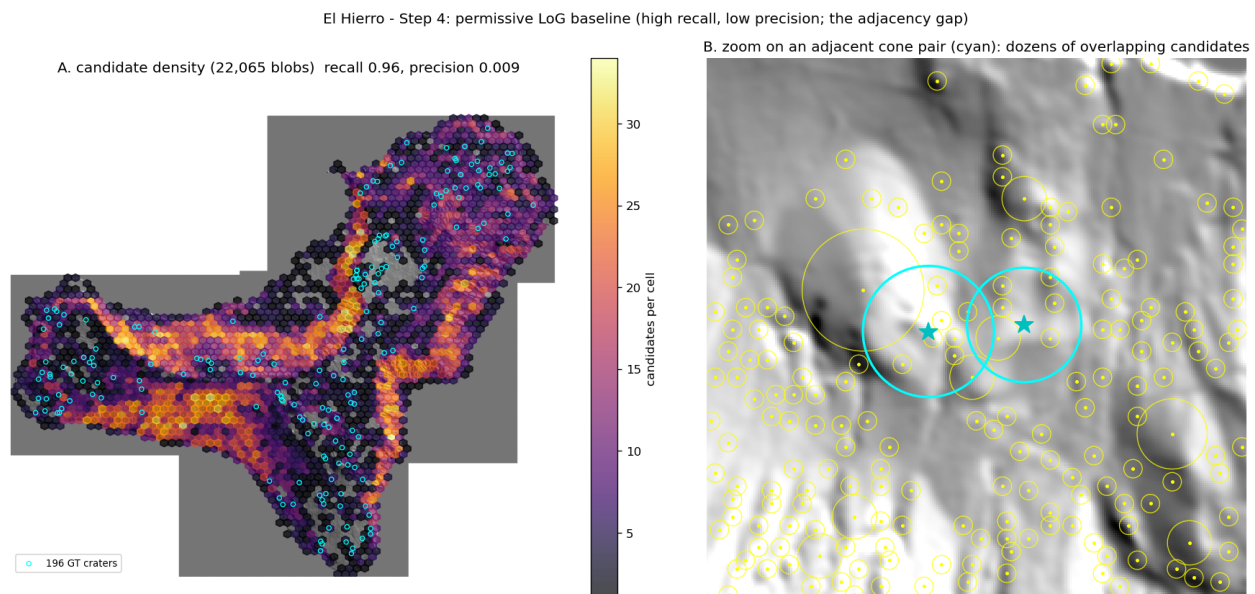


Figure 8: Step 4 on El Hierro. Left: density of the 22,065 permissive LoG candidates over the hillshade (recall 0.96, precision 0.009); the pool concentrates on steep terrain but is dominated by false positives. Right: zoom on an adjacent cone pair (cyan) surrounded by dozens of overlapping candidates, where no clean one-detection-per-cone assignment exists.

3.5 Step 5 — Watershed: separating adjacent cones (core contribution)

The LoG baseline finds the cones, but a point-based detector cannot split two cones that share a candidate: once a *one-to-one assignment* is enforced, crowded neighbours compete for the same blobs and 10 cones are lost.

We therefore switch to a **region-based** view.

The **watershed** transform floods the inverted relief from a set of markers and traces a boundary wherever two basins meet (deck 09_segmentation, p.8; SLIC_Superpixels, p.4), turning the detection into a per-region assignment.

The DEM is first **detrended** with a Difference-of-Gaussians high-pass, so that a small cone sitting on the volcano’s regional slope still becomes a local bump — otherwise only cones near the top of the island would be local maxima of absolute elevation.

The **markers** are the significant local maxima of the detrended relief (the cone summits), kept only inside the steep-terrain neighbourhood of the Step 3 mask; the watershed then assigns **one basin per summit**, so two adjacent cones separated by a saddle fall into two different basins.

Like the LoG pool, the watershed *over-segments* — it produces $\approx 4,400$ basins, mostly non-cone micro-relief — because separation, not precision, is the goal here; the Random Forest of Step 7 prunes the basins exactly as it prunes the LoG candidates.

What matters is the adjacency test: of the 10 cones the one-to-one LoG rule lost, 8 now sit in a basin separate from the neighbour they were competing with, and **3 of the 4** ground-truth adjacent pairs are split into distinct basins (Figure 9, right).

The one pair still merged is a *tightly nested* case — centroids only ≈ 10 px apart, one cone on the other’s flank, with no topographic saddle between them — which is handed to the multi-scale LoG of Step 6.

Approaches tried and discarded. A few variants were tested before settling on the method above.

Restricting the watershed to the steep mask alone fails: each crater centroid lies on the flat crater floor, *outside* the steep ring, so only $\approx 55/196$ cones land in any basin — hence we segment the whole island.

Detecting the markers with `peak_local_max` (label-constrained) stalls on the 7Mpx island, so the summits are found with a vectorised `maximum_filter` test instead.

Using the *raw* elevation for the markers misses cones on the volcano’s regional slope (they are not local maxima of absolute height): only 0–2 of the 4 adjacent pairs separate, which is what the Difference-of-Gaussians detrend fixes.

Filtering the markers by prominence (`h_maxima`) to cut their number re-merges close cones, because the saddle between them is shallow — so we keep plain local maxima and accept the over-segmentation, pruned later by the Random Forest exactly as for the LoG pool.

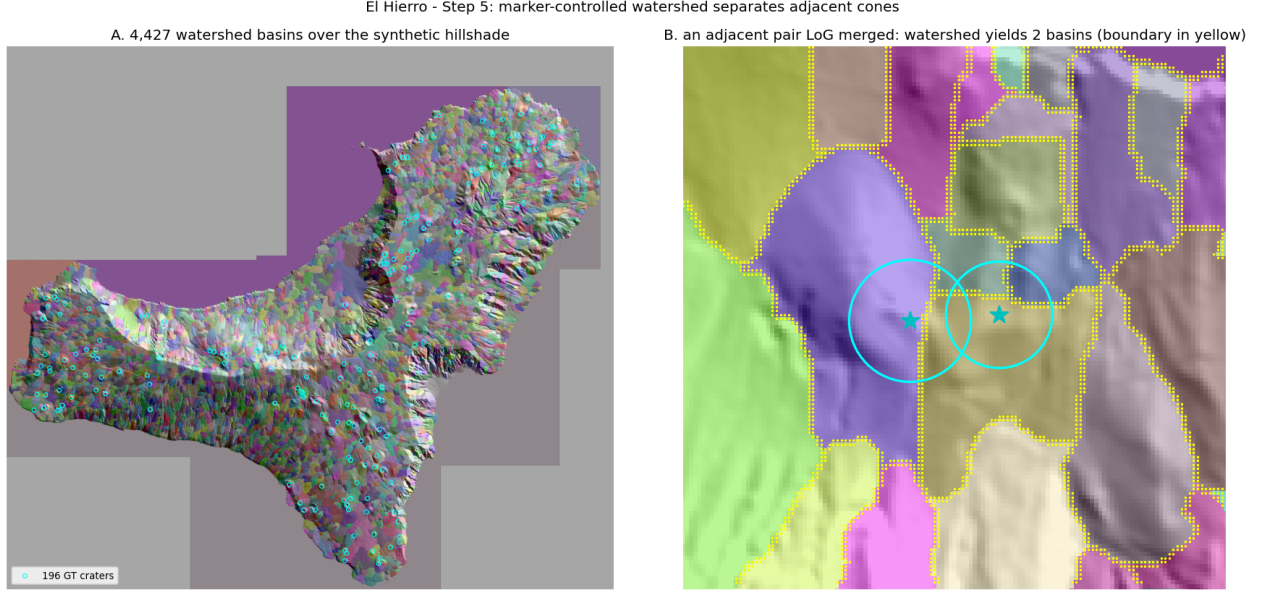


Figure 9: Step 5 on El Hierro. Left: the marker-controlled watershed partitions the island into basins, one per detected summit (it over-segments, as expected). Right: zoom on an adjacent cone pair (cyan) that the LoG baseline merged into a single detection; the watershed draws a boundary along the saddle (yellow) and assigns each crater its own basin.

3.6 Step 6 — Multi-scale LoG: separating nested cones (optional extension)

This step is optional: the core of the contribution is the watershed of Step 5, while Step 6 is a lighter, parallel add-on that covers the *other* way cones merge (nested by scale rather than adjacent in space) and can be skipped without breaking the pipeline.

The watershed separates cones that are side by side, but it cannot split a *nested* configuration: where a small cone sits inside or on the flank of a larger one there is a single mound and a single basin.

The LoG, being a **scale-space** detector, gives the complementary handle: the small cone and the large one peak at *different* scales σ (deck 03_image_matching, p.14–16), so they can be told apart by size at the same location.

The only change from the Step 4 baseline is the cross-scale pruning: the baseline used **overlap** = 0.3, which removes a blob whenever it overlaps a larger one by more than 30% — exactly what discards the smaller, nested response — whereas here we set **overlap** = 1.0 so that responses at different scales are **kept, not fused**.

Keeping the nested-scale responses enlarges the candidate pool (22,065 $\rightarrow$ 27,112) and **halves the one-to-one adjacency gap, from 10 to 5 cones**: crowded cones of different sizes no longer collapse onto a single blob, so each keeps its own scale-specific response (Figure 10).

Honest caveat. The recovered cones are a mix of genuinely nested/overlapping cases and cones that merely benefit from a denser pool, so the gain is real but not exclusively “nested”; the 5 still lost have no separable scale and remain the hard core.

A scale-aware match (forcing each blob’s scale to equal the crater radius) was also tried but made the gap far worse, because the ground-truth radii come from the open, noisy outlines — so we keep the plain assignment and let the multi-scale pool do the work.

Together, Steps 5 and 6 cover the two ways neighbours merge: the **watershed** separates cones that are side by side (by region), the **multi-scale LoG** separates cones that differ in size at the same place (by scale).

Both methods attack the same target — the 10 cones the baseline LoG one-to-one rule merges — so they are directly comparable on how many of those 10 they recover (Table 3): the **watershed of Step 5 is the stronger, core method** (8/10), while the optional multi-scale LoG of Step 6 recovers 5/10 and also covers the nested-by-scale case.

Method	separates	recovers (of 10)	own metric
Step 5 — watershed (by region)	adjacent cones	8/10	3/4 adjacent pairs split
Step 6 — multi-scale LoG (by scale) *	nested cones	5/10	one-to-one gap 10→5

Table 3: Separating merged cones: Step 5 vs Step 6, scored on the 10 cones the baseline LoG one-to-one rule loses. *Step 6 is the optional extension.

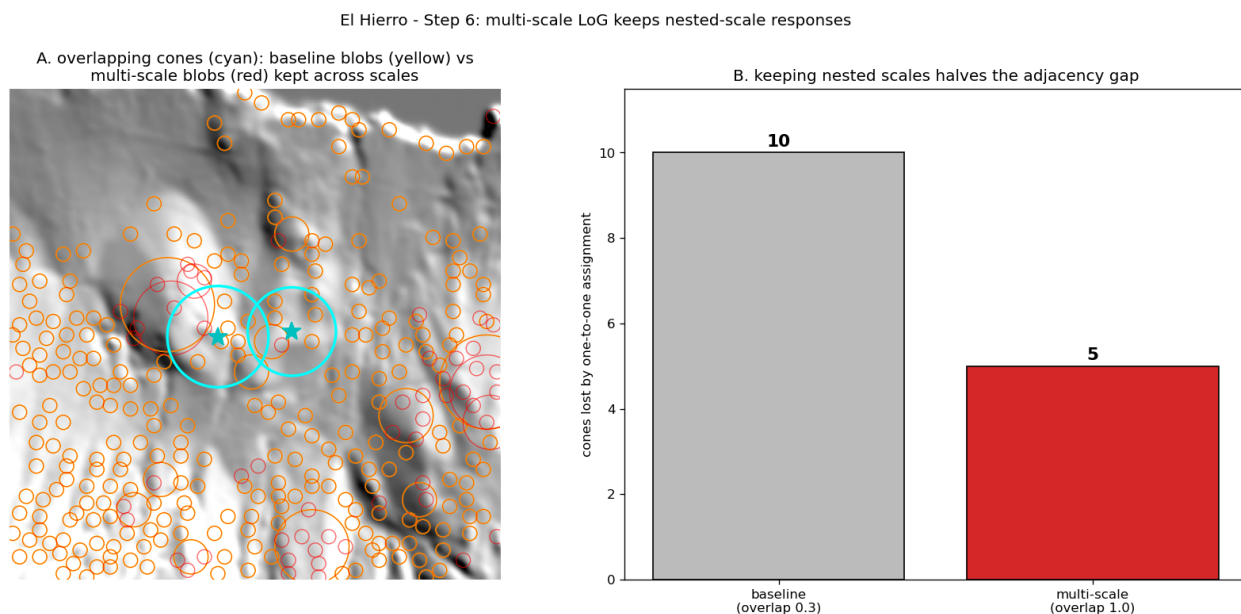


Figure 10: Step 6 on El Hierro. Left: zoom on an overlapping cone pair (cyan); the yellow circles are the Step 4 baseline blobs and the red circles the extra multi-scale responses kept by **overlap** = 1.0. Right: the strict one-detection-per-cone assignment loses 10 cones with the baseline pool but only 5 once nested scales are retained.